

1. Лабораторная работа №1. Изучение классов языка Си++

1.1. Цель и задачи работы, требования к результатам ее выполнения

Цель работы состоит в изучении основных понятий объектно- ориентированного программирования языка Си++ – классов и объектов, и овладении навыками разработки программ на языке Си++ с использованием объектно- ориентированных средств. Для достижения цели необходимо выполнить следующие задачи:

- изучить необходимые учебные материалы, посвященные основам объектно-ориентированного программирования на языке Си++;
- разработать программу на языке Си++ для решения заданного варианта задания;
- отладить программу;
- выполнить решение контрольного примера с помощью программы и ручной расчет контрольного примера;
- подготовить отчет по лабораторной работе.

1.2. Краткая характеристика объекта изучения

1.2.1. Понятие класса и объекта

Класс в языке Си++ – это новый тип, определяемый программистом, включающий данные (поля класса) и методы (функции) для обработки этих данных. Переменные этого типа называются объектами.

Формат объявления класса:

```
<Ключевое слово> <Имя_класса>  
{  
    <СПИСОК КОМПОНЕНТ>  
};
```

В качестве ключевого слова используется одно из трех ключевых слов:

- *struct*
- *class*
- *union*

Можно дать такое определение класса через структуру, которая была в языке Си. Класс – это структура, в которую введены методы для обработки полей.

Объекты – это переменные типа класса. Формат определения объектов:

<Имя_класса> <Имя_объекта1>, ... <Имя_объекта_N>;

Обращение к полям и методам класса внутри методов класса просто по имени, а за пределами класса через имя объекта и операцию «.» или через имя указателя на объект и операцию «->». Каждый объект класса имеет в оперативной памяти свои копии полей класса.

Пример работы с полями:

```
struct A { int i; void print() { printf("i=%d", i); } };  
A a1; A *pA=&a1;  
a1.i=10; a1.print();    pA->i=10; pA->print();  
a1.A::i=10; a1.A::print();    pA->A::i=10; pA->A::print();
```

1.2.2. Доступность компонент класса

Свойство доступности определяет возможность доступа к полям и методам за пределами класса (через имя объекта или через указатель на объект).

Существуют три статуса доступа:

- *public* (полностью доступны за пределами класса);
- *private* (не доступны за пределами класса, можно обращаться к компонентам только в методах своего класса);
- *protected* (доступны только в своем классе и в производных классах).

По умолчанию, если класс определен с ключевым словом *struct*, то все компоненты имеют статус доступа *public*. Если с ключевым словом *union*, тоже *public*, но все поля каждого объекта располагаются в памяти, начиная с одного адреса. Если класс определен с ключевым словом *class*, то все поля и методы по умолчанию имеют статус доступа *private*.

Статус доступа можно изменить с помощью соответствующих модификаторов, что продемонстрировано в следующем примере:

```
struct A  
{  
    ..... // Статус доступа public  
private:  
    ..... // Статус доступа private
```

```

protected:
.....// Статус доступа protected
};
class B
{
..... // Статус доступа private
public:
..... // Статус доступа public
protected:
.....// Статус доступа protected
};

```

1.2.3. Основные элементы класса

Компонентные данные и функции класса

Компонентные данные (поля класса) и функции класса (методы класса) уже во многом рассмотрены выше. Следует добавить, что методы класса могут быть определены внутри класса, в этом случае они по возможности (если нет ограничений) являются подставляемыми, но чаще всего класс содержит описание (заголовки) методов, а определения методов находятся за пределами класса. Эта возможность является удобной в проектах, состоящих из многих файлов. Создается отдельный файл с расширением .h (заголовочный файл), в котором находится описание класса вместе с полями и заголовками методов. Определения методов находится в файле реализации класса (файле с расширением .cpp). В этом случае, чтобы использовать класс в другом файле с исходным кодом, необходимо подключить заголовочный файл с описанием класса. Также внутри методов можно использовать умалчиваемые значения параметров (требования такие же, как к обычным функциям). В классах возможна перегрузка методов.

В следующем примере показаны перечисленные возможности:

```

struct Complex
{
    double real,    image; // Поля класса
    void define(double re=0.0, double im=0.0) // Определение метода
                                                // внутри класса
    {
        real=re; image=im; // Обращение к полям внутри метода
    }
}

```

```

    }
    void print(); // Описание метода
};

void Complex::print() // Определение метода за пределами класса
{
    printf("\nreal=%f image=%f", real, image);
}

```

Конструктор класса

Конструктор класса – специальный блок операторов (инструкций), вызываемый при создании объекта. Назначение: присвоение начальных значений полям, выделение памяти, открытие файлов, сетевых соединений и т.п. Имя конструктора совпадает с именем класса, конструктор не имеет возвращаемого значения. Возможна перегрузка конструкторов. Конструктор может определяться как внутри класса, так и за пределами.

Формат определения конструктора внутри класса:

```

Имя_класса(Список_формальных_параметров)
{ Операторы_тела_конструктора }

```

По умолчанию класс всегда имеет конструктор копирования вида $A(A\& a) \{ \dots \}$ (A – имя класса), создающий копию объекта (происходит копирование полей), и если нет явного конструктора, то по умолчанию создается конструктор без параметров. Эти конструкторы можно переопределять.

Примеры вызовов конструкторов:

```

A a1; A * pA=new A; // Вызываются конструкторы без параметров
A a2(3, 4); A * pA2=new A(3, 4);
// Вызываются конструкторы с 2-мя параметрами

```

Для конструктора с одним параметром можно использовать форму:

```

A a1=5; A a2=a1; вместо A a1(5); A a2(a1);

```

Деструктор класса

Деструктор – специальный блок операторов (инструкций), служащий для деинициализации объекта (освобождение памяти, закрытие файлов и т.п.).

Вызывается автоматически при удалении объекта, например, оператором *delete* или при выходе из блока, в котором существует объект. Не имеет возвращаемого значения и параметров. Может определяться как внутри класса, так и за пределами. Пример деструктора:

```

~Имя_класса()

```

```
{  
    тело_деструктора  
}
```

1.3. Задачи и порядок выполнения работы

В работе необходимо разобраться с понятием класса некоторой предметной области и соответствующему ему классу как типу языка Си++, введенного пользователем. Также необходимо знать отличие классов от объектов и назначение и порядок использования основных элементов класса в Си++: полей, методов (функций класса), конструкторов, деструктора. Обратить внимание на способы создания массива объектов класса динамически.

Студент разрабатывает программу на языке Си++ в виде консольного приложения, в программе необходимо создать **динамический массив и последовательной контейнер STL (vector, deque или list) объектов некоторого класса**, данные об объектах читаются из текстового файла. Результаты работы программы и результаты ручного расчета (при необходимости) представляет преподавателю в отчете.

Условие задачи:

Класс автомобиль. Содержит поля (статус доступа private):

- название (марка);
- средний расход топлива (в литрах на 100 км).

Методы и конструкторы:

- конструктор для инициализации полей;
- метод для расчета требуемого объема топлива для заданного пробега (метод имеет 1 параметр, который задает требуемый пробег в километрах, возвращает требуемый объем топлива в литрах);
- метод для печати параметров объекта.

При необходимости можно использовать другие методы и конструкторы.

Требуется: создать массив и vector объектов класса, рассчитать требуемый суммарный объем топлива для заданного пробега всех автомобилей, пробег вводится с клавиатуры, вывести на печать параметры объектов и рассчитанный объем топлива.

Пример выполнения работы

Следует отметить, что для создания массива объектов динамически, когда существует конструктор с параметрами для инициализации полей объекта, можно использовать два основных способа.

Особенности первого способа:

- необходимо в класс включить конструктор без параметров и отдельный метод для инициализации полей, например, с именем *set* и числом параметров, которые соответствуют числу инициализируемых полей класса;

- объявляем указатель на массив объектов, если имя класса *MyClass*, то это объявление может иметь вид: *MyClass *pOb*;

- задаем число объектов, например, вводим с клавиатуры или читаем из файла как значение целой переменной *n*, и создаем массив объектов динамически *pOb=new MyClass[n]*; здесь, при создании каждого объекта, вызывается конструктор без параметров, именно для этого его и нужно включить в класс;

- в цикле для каждого объекта вводим параметры или читаем из файла для его инициализации и инициализируем уже созданный объект с помощью вызова метода *set*, это действие может иметь вид *pOb[i].set(...)*; где в метод *set* вместо многоточия передаются фактические параметры.

Особенности второго способа (здесь не нужен конструктор без параметров и метод *set*):

- объявляем указатель на указатель на массив объектов, если имя класса *MyClass*, то это объявление может иметь вид: *MyClass **ppOb*;

- задаем число объектов, например, вводим с клавиатуры или читаем из файла как значение целой переменной *n*, и создаем массив указателей динамически *ppOb=new MyClass*[n]*;

- в цикле для каждого объекта вводим параметры или читаем их из файла для его инициализации и создаем объект динамически с помощью вызова конструктора с параметрами, это действие может иметь вид *ppOb[i]=new MyClass(...)*; где вместо многоточия передаются фактические параметры для конструктора класса.

При использовании последовательного контейнера (*vector*, *deque* или *list*) объект можно добавлять в конец контейнера с помощью метода *push_back*, или сразу создавать контейнер из *n* объектов и далее инициализировать каждый элемент с помощью метода *set*.

Ниже в листинге показаны оба способа, один из способов представлен в виде комментариев. В листинге данные вводятся с клавиатуры, в работе их надо читать из файла.

Листинг программы с комментариями:

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
class Avt // Класс автомобиль
{
    string marka; // Марка
    double rash; // Расход топлива на 100 км
public:
    Avt() // Конструктор без параметров создает "пустой" объект
    {
        rash = 0;
        marka = "";
    }
    void set(string mar, double r) // Функция для инициализации полей для
        // созданного "пустого" объекта
    {
        marka= mar; // Копируем строку, содержащую марку автомобиля
        rash = r; // Задаем значение расхода топлива на 100 км
    }
    Avt(string mar, double r) // Конструктор для инициализации полей
    {
        marka=mar; // Копируем строку, содержащую марку автомобиля
        rash = r; // Задаем значение расхода топлива на 100 км
    }
    double getRash(double dlina) const // Функция возвращает - сколько нужно топлива
для
        // пробега заданного расстояния
    {
        return rash * dlina / 100.;
    }
    void print() const // Функция для печати полей объекта
    {
        cout << "\nmarka: " << marka << " rashod na 100 km=" << rash;
    }
};

int main()
{
    int n; // Известное число объектов
    cout << "n="; cin >> n; // Ввод с клавиатуры n
    double rast; // Расстояние, для которого требуется вычислить расход топлива
    cout << "rast="; cin >> rast; // Ввод с клавиатуры расстояния
    double SumRashod = 0; // Суммарный расход топлива для всех автомобилей
    string str; double r; // Вспомогательные переменные для ввода марки
    // автомобиля и расхода топлива

    /*****
    Первый способ создаем массив "пустых" объектов и инициализируем их с
    помощью функции set
    *****/

    Avt *pAvt; // Указатель на массив
    pAvt=new Avt[n]; // Для каждого объекта вызывается конструктор без параметров,
    // т.е. созданы "пустые" объекты
    vector<Avt> vecAvt(n); // Создаем пустые объекты, вызывается конструктор без
параметров
    // Цикл ввода данных для объектов
    for(int i=0; i<n; i++)
    {
        cout<<"Object N="<<(i+1)<<":\n"<<"marka: ";
        cin.ignore();
    }
}
```

```

getline(cin, str); // Ввод марки автомобиля
cout<<"Rashod="; cin>>r; // Ввод расхода топлива
pAvt[i].set(str, r); // Вызываем функцию set для инициализации полей
vecAvt[i].set(str, r); // Инициализация для vector
// объектов
}
// Цикл печати полей для объектов
for(int i=0; i<n; i++) pAvt[i].print();
// Цикл для расчета суммарного расхода топлива
for(int i=0; i<n; i++) SumRashod+=pAvt[i].getRash(rast);
cout << "\nSumRashor=" << SumRashod; // Вывод на печать суммарного расхода
топлива
// Тоже делаем для vector
SumRashod = 0;
// Цикл печати полей для объектов
for (const auto pos : vecAvt) pos.print();
// Цикл для расчета суммарного расхода топлива
for (const auto pos : vecAvt) SumRashod += pos.getRash(rast);
cout << "\nSumRashor=" << SumRashod; // Вывод на печать суммарного расхода
топлива
/*****
Конец первого способа
*****/

/*****
Второй способ создаем массив указателей на объекты и
далее каждый объект создается динамически с помощью конструктора с инициализацией
*****/

/*Avt **ppA; // Указатель на массив указателей
ppA = new Avt*[n]; // Создаем массив указателей
vector<Avt> vecAvt; // Параллельно создаем вектор автомобилей
for (int i = 0; i < n; i++)
{
    cout << "Object N=" << (i + 1) << ":\n" << "marka: ";
    cin.ignore();
    getline(cin, str); // Ввод марки автомобиля
    cout << "Rashod="; cin >> r; // Ввод расхода топлива
    ppA[i] = new Avt(str, r); // Создание объекта динамически
    // с вызовом конструктора с параметрами
    vecAvt.push_back(Avt(str, r)); // Добавляем объект в контейнер
}
// Цикл печати полей для объектов
for (int i = 0; i < n; i++) ppA[i]->print();
// Цикл для расчета суммарного расхода топлива
for (int i = 0; i < n; i++) SumRashod += ppA[i]->getRash(rast);
cout << "\nSumRashor=" << SumRashod; // Вывод на печать суммарного расхода
топлива
// Тоже делаем для vector
SumRashod = 0;
// Цикл печати полей для объектов
for (const auto pos : vecAvt) pos.print();
// Цикл для расчета суммарного расхода топлива
for (const auto pos : vecAvt) SumRashod += pos.getRash(rast);
cout << "\nSumRashor=" << SumRashod; // Вывод на печать суммарного расхода
топлива
/*****
Конец второго способа
*****/
return 0;
}

```

1.4. Форма отчета по лабораторной работе

Отчет должен содержать: титульный лист, цель работы, условие задачи, текст программы с комментариями, ручной расчет контрольного примера и результаты решения контрольного примера программой для проверки правильности работы алгоритма, выводы по работе.

При выполнении работы все входные данные читаются из текстового файла `input.txt` (создать этот файл любым текстовым редактором), результаты выводятся на консоль и в файл `output.txt`, для этого использовать одну функцию для вывода. В отчете представить содержимое этих файлов. Для задания строк можно использовать `std::string`.

1.5. Вопросы для самоконтроля

1. Классы и объекты.
2. Статусы доступа полей и методов.
3. Назначение и перегрузка конструкторов.
4. Деструктор класса.

Варианты заданий для лабораторной работы № 1

Во всех вариантах требуется следующее:

Описать класс, включающий заданные поля и методы (функции). Разработать программу (или две программы, выбрать по желанию), которая **создает динамический массив объектов и последовательный контейнер STL (vector, deque или list выбрать самостоятельно) объектов** и выполняет требуемые действия. **Все исходные данные для работы программы читаются из текстового файла, созданного в простом редакторе типа «Блокнот». Выходные данные выводятся на консоль и в текстовый файл, для этого использовать одну функцию для вывода.**

Вариант 1

Класс – сотрудник предприятия. Параметры (поля класса) – ФИО, оклад, надбавка к окладу в %. Статус доступа всех полей private. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров, функцию вычисления зарплаты (зарплата = оклад + процентная надбавка от оклада). Вывести на печать параметры всех сотрудников и суммарную их зарплату.

Вариант 2

Класс – студент. Параметры (поля класса) – ФИО, массив из m-х оценок за последнюю сессию. Статус доступа всех полей private. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров, функцию проверки возможности получения студентом стипендии (все оценки без троек). Вывести на печать всех студентов, получающих стипендию.

Вариант 3

Класс – банковский вклад. Параметры (поля класса): ФИО владельца, текущая сумма, годовая процентная ставка (проценты начисляются ежегодно с капитализацией начисленных процентов с основной суммой). Статус доступа всех полей private. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров, функцию расчета суммы на счету через заданное число лет (число лет – параметр функции). Вывести на печать параметры всех вкладов и суммарную сумму на счетах через заданное число лет, которое вводится с клавиатуры.

Вариант 4

Класс – аппаратно- программное средство защиты (СЗ) от несанкционированного доступа (НСД). Параметры (поля класса) – название и номер класса защищенности от НСД (Существует семь классов защищенности от НСД, наивысший 1-ый, самый низкий 7, например, если требуется обеспечить защищенность по 3- му классу, то можно использовать СЗ с классами 1, 2 или 3). Статус доступа всех полей private. Класс включает: конструктор, при необходимости функции доступа к полям, функцию, проверяющую можно ли это СЗ использовать для заданного класса (номер заданного класса – параметр функции), функцию печати параметров СЗ. Вывести на печать параметры тех СЗ, которые можно использовать для заданного класса защищенности, номер класса защищенности вводится с клавиатуры.

Вариант 5

Класс – криптографический метод защиты информации. Параметры (поля класса) – название и тип (симметричный или несимметричный). Статус доступа всех полей private. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров. Вывести на печать все методы заданного типа, тип вводится с клавиатуры.

Вариант 6

Класс – прямоугольник. Параметры (поля класса) – длина, ширина. Статус доступа всех полей private. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров, функцию вычисления площади. Вывести на печать все параметры прямоугольников, площади которых превышают заданное значение, которое вводится с клавиатуры.

Вариант 7

Класс – автомобиль. Параметры (поля класса) – марка, максимальная скорость (км/ч). Статус доступа всех полей private. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров. Вывести на печать параметры тех автомобилей, максимальная скорость которых превышает заданное значение, введенное с клавиатуры.

Вариант 8

Класс – квадратное уравнение $ax^2 + bx + c = 0$ ($a \neq 0$). Параметры (поля класса) – a , b , c . Статус доступа всех полей `private`. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров, функцию расчета дискриминанта. Вывести на печать параметры тех уравнений, которые имеют вещественные корни.

Вариант 9

Класс – полином $a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$ ($a_n \neq 0$). Параметры (поля класса): n и массив коэффициентов $a_n, a_{n-1}, \dots, a_0$. Статус доступа всех полей `private`. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров, функцию вычисления значения полинома при заданном x (x - параметр функции). Вывести на печать параметры всех полиномов и сумму их значений при заданном значении x , вводимом с клавиатуры.

Вариант 10

Класс - книга. Параметры (поля): автор, название, количество страниц. Статус доступа всех полей `private`. Класс включает: конструктор, функцию печати параметров, при необходимости функции доступа к полям. Распечатать параметры книги с максимальным количеством страниц.

Вариант 11

Класс – межсетевой экран (МЭ). Параметры (поля класса) – название и номер класс защищенности (Существует пять классов защищенности МЭ, наивысший 1-ый, самый низкий 5, например, если требуется использовать МЭ 3- го класса защищенности, то можно использовать МЭ с классами 1, 2 или 3). Статус доступа всех полей `private`. Класс включает: конструктор, при необходимости функции доступа к полям, функцию, проверяющую можно ли МЭ использовать для заданного класса (номер заданного класса – параметр функции), функцию печати параметров МЭ. Вывести на печать параметры тех МЭ, которые можно использовать для заданного класса защищенности, номер класса защищенности вводится с клавиатуры.

Вариант 12

Класс – персональный компьютер (ПК). Параметры (поля класса): название процессора, тактовая частота. Статус доступа всех полей `private`. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров.

Вывести на печать параметры всех компьютеров в порядке невозрастания тактовой частоты.

Вариант 13

Класс – программа- антивирус. Параметры (поля класса): название, число вредоносных программ в базе. Статус доступа всех полей private. Класс включает: конструктор, при необходимости функции доступа к полям, функцию печати параметров. Вывести на печать параметры антивирусов и антивирус с самой большой базой вредоносных программ.

Вариант 14

Класс - вещественное число, записанное с точкой. Параметры - его значение (вещественный тип) и запись (строковое представление числа в десятичной системе счисления с точкой). Статус доступа всех полей private. Класс включает: конструктор, при необходимости функции доступа к полям, функцию, определяющую количество цифр в целой части числа в десятичной записи, функцию печати параметров. Напечатать все числа, сумму введенных чисел и суммарное количество цифр в целых частях всех чисел.

Вариант 15

Класс - предложение. Параметры: массив слов ($n < 10$) и их количество. Статус доступа всех полей private. Класс включает: конструктор и функцию, определяющую количество слов, длиннее 5 букв, при необходимости функции доступа к полям, функцию печати параметров. Напечатать параметры предложений и процент слов длиннее 5 букв в заданном тексте.

Вариант 16

Класс - выражение, состоящее из целых чисел и знаков операций (скобок нет).

Параметры: массив значений чисел ($n < 10$), количество чисел и массив знаков операций (тип char). Статус доступа всех полей private. Класс включает: конструктор и функцию, вычисляющую результат (приоритеты операций не учитывать, считать приоритет операций одинаковым), при необходимости функции доступа к полям, функцию печати параметров. Ввести несколько выражений и вывести результаты в порядке, обратном вводу.

Вариант 17

Класс - некоторый товар в магазине. Параметры: наименование, количество и стоимость. Статус доступа всех полей private. Класс включает: конструктор и функцию, определяющую суммарную стоимость товара, при необходимости функции доступа к полям, функцию печати параметров. Напечатать параметры всех товаров и суммарную стоимость всех товаров в магазине.

Вариант 18

Класс - некоторый товар в магазине. Параметры: наименование, количество и закупочная цена. Статус доступа всех полей private. Класс включает: конструктор и функцию, определяющую стоимость товара исходя из заданного процента прибыли (процент прибыли – параметр функции), при необходимости функции доступа к полям, функцию печати параметров. Напечатать параметры всех товаров и суммарную стоимость всех товаров в магазине с учетом заданного процента прибыли, который вводится с клавиатуры.

Вариант 19

Класс - студент. Параметры (поля): ФИО, массив экзаменационных оценок ($m=4$). Статус доступа всех полей private. Класс включает: конструктор и функцию определения среднего балла, при необходимости функции доступа к полям, функцию печати параметров. Напечатать параметры всех студентов и 3-х самых сильных студентов группы.

Вариант 20

Класс - ангар. Параметры (поля): ширина и длина. Статус доступа всех полей private. Класс включает: конструктор и функцию, определяющую площадь помещения, при необходимости функции доступа к полям, функцию печати параметров. Напечатать параметры ангаров и площадь склада, состоящего из этих ангаров.

Вариант 21

Класс - квартира. Параметры (поля): общая площадь и стоимость одного квадратного метра. Статус доступа всех полей private. Класс включает: конструктор и функцию, определяющую стоимость квартиры, при необходимости функции доступа к полям, функцию печати параметров. Напечатать параметры всех квартир и все квартиры, стоимость которых не превышает заданной суммы (сумма вводится с клавиатуры).

Вариант 22

Класс - квартира. Параметры (поля): стоимость и количество комнат. Статус доступа всех полей private. Класс включает: конструктор и функцию, определяющую среднюю стоимость одной комнаты, при необходимости функции доступа к полям, функцию печати параметров. Напечатать параметры всех квартир и все квартиры, стоимость одной комнаты в которых не превышает заданной суммы (сумма вводится с клавиатуры).

Вариант 23

Класс - выставочные экспонаты. Параметры (поля): название, время экспонирования (в днях), стоимость одного дня экспонирования. Статус доступа всех полей private. Класс включает: конструктор и функцию определения стоимости экспонирования, при необходимости функции доступа к полям, функцию печати параметров. Вывести на печать параметры экспонатов и экспонат, стоимость экспонирования которого максимальна.

Вариант 24

Класс - книга. Параметры (поля): автор, название, количество экземпляров и количество желающих ее прочитать читателей. Статус доступа всех полей private. Класс включает: конструктор и функцию определения средней длины очереди на чтение каждого экземпляра, при необходимости функции доступа к полям, функцию печати параметров. Напечатать параметры книг и наиболее читаемую книгу в библиотеке.

Вариант 25

Класс - выражение, состоящее из целых чисел и знаков операций (скобок нет).

Параметры (поля): строка, содержащая выражение. Статус доступа всех полей private. Класс включает: конструктор и функцию, определяющую количество операций, при необходимости функции доступа к полям. Ввести несколько выражений и определить суммарное количество операций в них.

Вариант 26

Класс – вектор на плоскости. Параметры (поля): координаты конца вектора: x , y (начало вектора в точке с координатами $0, 0$), Статус доступа всех полей private. Класс включает: конструктор, функцию печати параметров, при необходимости функции

доступа к полям, функцию вычисления длины вектора. Вывести на печать все вектора и вектор с наибольшей длиной.

Вариант 27

Класс - скаковая лошадь. Параметры: кличка и массив рекордов, содержащий 5 лучших результатов, показанных лошадью на скачках (результат определяется временем). Статус доступа всех полей `private`. Класс включает: конструктор и функцию, определяющую среднее время, показанное лошадью, при необходимости функции доступа к полям, функцию печати параметров. Вывести на печать параметры лошадей и среднее время по всей конюшне.